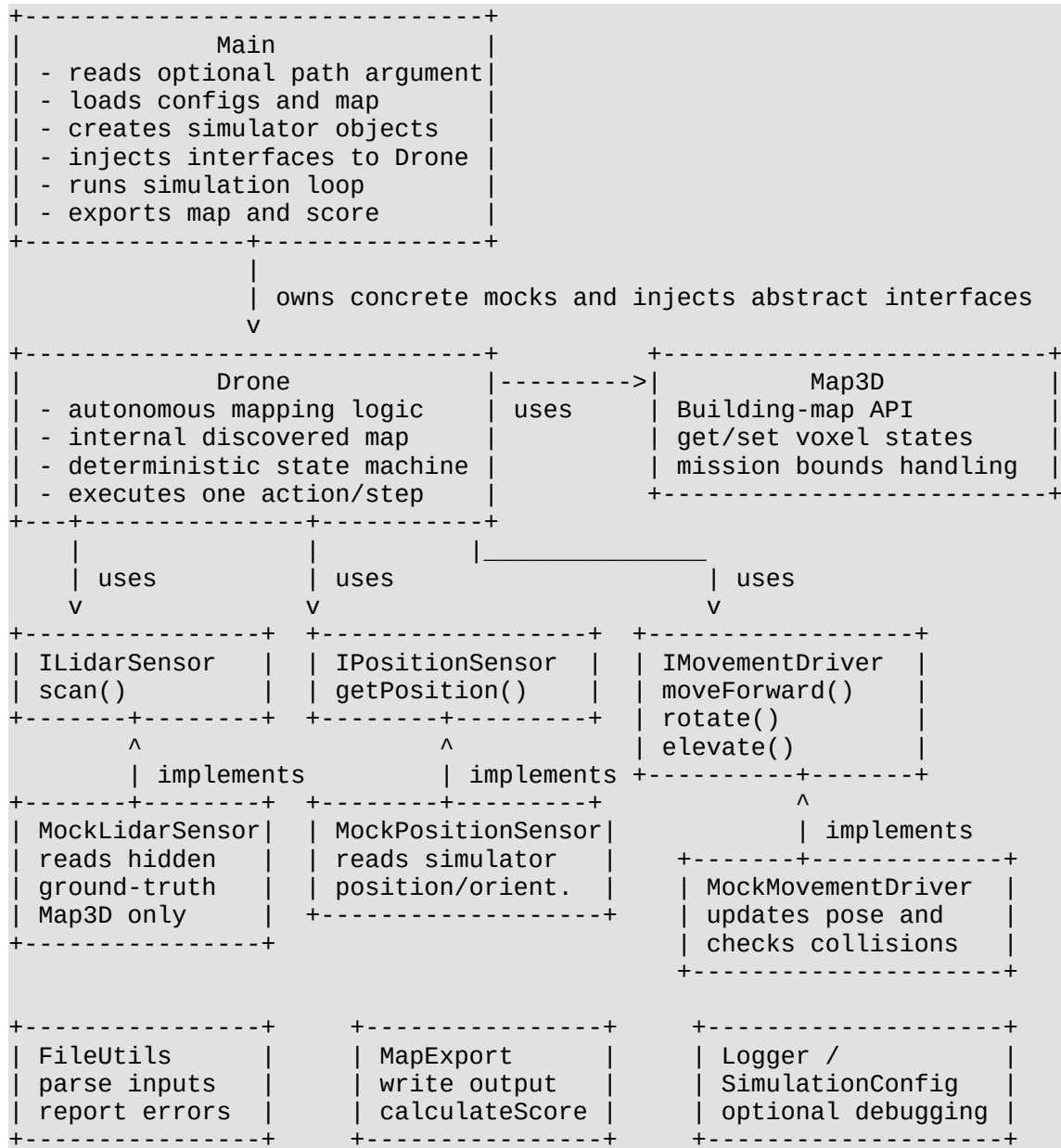


HW1 - Drone Mapping System

High Level Design

1. UML Class Diagram



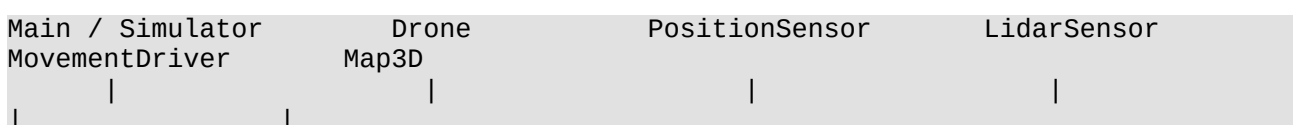
Main roles

Component	Responsibility
Main	Composition root and simulation manager. It loads input files, creates the mock simulator objects, injects them into Drone through interfaces, runs the outer loop, writes outputs,

Component	Responsibility
	and prints/saves the final score.
Drone	The autonomous algorithmic unit. It owns the discovered map, decides the next action, and executes exactly one action per <code>executeNextAction()</code> call through <code>ILidarSensor</code> , <code>IPositionSensor</code> , and <code>IMovementDriver</code> . It never receives the hidden building map.
Map3D	Building-map abstraction/API. Stores voxel values: 0 free, 1 occupied, -1 unknown, -2 outside mission bounds.
ILidarSensor	Interface for LiDAR providers. Implemented by <code>MockLidarSensor</code> .
IPositionSensor	Interface for pose providers. Implemented by <code>MockPositionSensor</code> .
IMovementDriver	Interface for movement executors. Implemented by <code>MockMovementDriver</code> .
MockLidarSensor	Reads the hidden ground-truth <code>Map3D</code> and returns only <code>ScanResult</code> values to the drone.
MockPositionSensor	Exposes the current simulator pose to the drone through the position interface.
MockMovementDriver	Updates simulator pose and checks that the spherical drone does not collide with occupied or out-of-bounds voxels.
FileUtils	Parses <code>drone_config.txt</code> , <code>mission_config.txt</code> , and <code>map_input.txt</code> ; reports recoverable input errors to <code>Main</code> .
MapExport	Writes <code>map_output.txt</code> and calculates the final score. <code>Main</code> writes <code>score.txt</code> .
Logger	Optional logging utility controlled by <code>simulation_config.txt</code> . It is separated from the algorithm and does not affect decisions.

2. UML Sequence Diagram - Main Flow

The sequence focuses on the runtime mapping loop. File parsing happens before the loop and is intentionally not expanded here.



```

| create mocks | | | | |
| create Drone and inject interfaces | | |
| ----->| | |
| | | | | |
| executeNextAction() | | |
| ----->| | |
| | getNextCommand() | | |
| | ----->| internal state machine decision
| | <-----| | | |
| | | | | |
| | [GetLocation] getPosition() | | |
| | ----->| | |
| | <-----| Pose3D | | |
| | updatePose(pose) | | |
| ----->|
| | mark current voxel as FREE | | | |
| | | | | |
| | [Scan] scan(position, azimuth, elevation) | | |
| | ----->| | |
list | | | <-----| ScanResult
| | | processScanResults(results) | | |
| ----->|
| | update discovered map with FREE / OCCUPIED voxels | | | |
| | | | | |
| | [Rotate / Advance / Elevate] execute movement | | |
| ----->| | |
<-----| success/fail |
| | getPosition() after successful movement | | | |
| | ----->| | |
| | <-----| updated Pose3D | | |
| | updatePose(pose) | | |
| <-----| executed DroneCommand with metadata

```

```

|   | log command / count iteration / stop on Finished or collision
|   |
|   |   |
|   |   |
|   |   | repeat until DroneCommandType::Finished or safety limit
|   |   |
|   |   | export drone Map3D and score against hidden ground-truth map
|   |   |

```

The important architectural point is that `Main` does not call `scan()`, `moveForward()`, `rotate()`, `elevate()`, or `getPosition()` directly during the loop. It only calls `Drone::executeNextAction()`. The drone executes the selected action through the injected interfaces and returns the executed `DroneCommand` so that `Main` can log and control the simulation.

3. Design Considerations and Alternatives

3.1 Separation between algorithm and simulator truth

The most important design decision is strict separation between the drone algorithm and the real building map. The hidden ground-truth `Map3D` is owned by the simulator side and is used only by the mock LiDAR, the mock movement driver, and the scoring function. The `Drone` never receives this map directly.

The drone updates its internal discovered map only from:

- `ScanResult` values returned by `ILidarSensor`.
- pose values returned by `IPositionSensor`.
- success/failure information returned by `IMovementDriver`.

This preserves the assignment requirement that `map_input.txt` is visible only to the simulated sensor/driver environment and not to the drone brain.

3.2 Interfaces and dependency injection

The concrete simulator objects are created in `Main`, but the drone receives only abstract interfaces:

- `ILidarSensor` for scan data.
- `IPositionSensor` for current pose.
- `IMovementDriver` for physical movement execution.

This keeps the mocks transparent to the drone. The drone is written against interfaces, so the same algorithm could later be tested with other sensor or movement implementations without changing the drone logic.

`Main` is therefore a composition root: it owns the concrete objects and wires them together. The drone is the autonomous unit that performs one action per call to `executeNextAction()`.

3.3 Strong types with `mp-units`

Distances and angles are represented with `mp-units` quantities, mainly centimeters and degrees. Input files use centimeters for distances and degrees for angles. In the code, using strong unit types helps prevent accidental mixing of distances and angles, such as passing an angle where a distance is expected.

3.4 Map representation

`Map3D` exposes the required 3D map API and returns the required values:

- 0 for free space.
- 1 for occupied space.
- -1 for unknown / not mapped space.
- -2 for coordinates outside the required mission boundaries.

Internally, the implementation uses one contiguous `std::vector<int>` and converts (x, y, z) coordinates into a single index. This gives simple ownership, predictable memory layout, O(1) access, and straightforward export to `map_output.txt`.

A nested `vector<vector<vector<int>>>` was rejected because it adds allocation overhead and pointer indirection. A hash-based sparse matrix was considered; however, for Exercise 1 the map sizes are small enough that a flat vector keeps the implementation simpler, deterministic, and easier to inspect. The public API still behaves as a 3D map abstraction.

3.5 Movement and collision safety

The drone is modeled as a perfect sphere. Its radius is derived from the configured minimum pass dimensions. The movement driver checks the hidden ground-truth map before accepting movement. It rejects positions where the drone sphere would touch an occupied voxel or leave the legal map area.

This separates planning from physical validation: the drone plans using its discovered map and safety rules, while the simulator driver provides the final physical collision check against the hidden map.

3.6 Deterministic exploration algorithm

The exploration algorithm is deterministic. It uses a fixed scan-plan-move cycle and does not use random functions. The drone scans fixed directions, updates its internal map, first tries a local sweep to nearby safe unvisited cells, and then uses BFS frontier planning when local movement is blocked.

This makes the behavior repeatable, easier to debug, and consistent with the requirement that the algorithm must not depend on randomness.

3.7 Exercise 1 finish condition

The original high-level mission describes a return-to-home condition. For Exercise 1, the assignment explicitly allows the program to report `Finished` when mapping is complete, without

physically returning to the starting position. Therefore, the implementation finishes when reachable frontiers are exhausted and no additional useful scan/move action is available.

3.8 Alternatives considered

Topic	Alternative	Chosen design and reason
Object ownership	Owning raw pointers inside <code>Drone</code>	Rejected. Concrete mocks are owned by <code>Main</code> ; <code>Drone</code> stores non-owning interface pointers/references.
Sensor access	Let <code>Main</code> execute every command directly	Rejected after refactoring. <code>Main</code> now only calls <code>Drone::executeNextAction()</code> , while <code>Drone</code> uses the injected interfaces.
Map storage	Nested vectors or hash map	Flat <code>std::vector<int></code> for simple indexing, cache locality, deterministic traversal, and easy output.
Hidden map access	Let <code>Drone</code> read the ground-truth map	Rejected because it violates strict input separation.
Exploration	Random walk	Rejected because the assignment requires deterministic behavior. Sweep + BFS frontier planning was chosen.
File format	Binary map file	Plain text was chosen because it is readable, debuggable, and easy to compare.

4. Implementation Details

4.1 Command-driven autonomous step

The drone contains a state machine with the states `Scanning`, `Planning`, `Moving`, and `Finished`.

`Drone::getNextCommand()` decides what the next logical command should be.

`Drone::executeNextAction()` then executes exactly one command through the injected interfaces and returns the executed `DroneCommand` to `Main`.

This gives a clean separation:

- `Drone` decides and executes autonomous actions using sensors and movement interfaces.
- `Main` manages the outer simulation loop, logging, iteration limit, collision/failure handling,

and output files.

4.2 Scan command defaults

A scan command can include explicit azimuth and elevation angles. If no scan angles are supplied, the default is:

- azimuth relative to the current drone direction: 0 degrees.
- elevation angle: 0 degrees.

This means the default scan points forward in the drone's current direction.

4.3 LiDAR results

The LiDAR result structure stores, for each beam, the beam azimuth/elevation information and the detected distance. If a hit is below the configured `Z-min`, the distance is reported as 0, representing a hit that is too close to measure accurately.

4.4 Exploration algorithm

At each mapped position, the drone scans a fixed set of directions: forward, right, back, left, up, and down. Horizontal directions are represented by azimuth angles, and vertical scans are represented by elevation angles.

Scan results update the internal discovered map by marking cells along each ray as `FREE` and valid hit cells as `OCCUPIED`. After scanning, the drone first attempts a local sweep to a nearby safe unvisited cell. If no local sweep move is available, it runs BFS over known-free navigable cells to find the nearest frontier.

The planner first searches for a strict `Frontier6`, which is a free cell adjacent to unknown space in one of the six cardinal directions. If no such frontier is reachable, it falls back to `Frontier26`, which also considers nearby unknown cells in a Moore neighborhood.

If a planned move is blocked because an unknown voxel lies inside the drone's safety sphere, the drone performs a targeted scan toward that unknown voxel before giving up on the path.

4.5 Movement execution

Movement commands are emitted as `Rotate`, `Advance`, or `Elevate`. Negative values are supported by the movement interfaces, allowing reverse movement or downward elevation when needed. The movement driver enforces the maximum allowed movement per command and validates that the drone can safely occupy the target position.

After successful movement, `Drone::executeNextAction()` asks the position sensor for the updated pose and updates the internal position estimate.

4.6 Mission bounds

Mission boundaries are parsed from `mission_config.txt` and stored in the map configuration. Coordinates outside the valid area return `OUT_OF_BOUNDS (-2)`. Writes outside the valid area are ignored. This ensures that both the internal map and the simulator map respect the

required mapping boundaries.

4.7 Resolution handling

Exercise 1 supports a single map resolution. If the mission requests an unsupported resolution, the program recovers by using the supported default resolution and writes a recoverable input error to `input_errors.txt`.

4.8 Recharging positions

The mission parser supports a list of recharging positions, including an empty list. These positions are parsed and stored to satisfy the required mission-file format. They are not used by the navigation algorithm in Exercise 1 because the battery is assumed to be infinite.

4.9 Map output and scoring

`MapExport` writes the discovered map into `map_output.txt` using the same text format as `map_input.txt`, so the two maps can be compared. The output file is overwritten if it already exists.

The scoring function compares the discovered map with the hidden ground-truth map and produces a score between 0 and 100. The current implementation focuses on comparing mapped free/occupied cells in the map. A future improvement is to exclude completely unreachable sealed areas from the denominator using a reachability search from the start position.

4.10 Error handling

Recoverable input issues are collected and written to `input_errors.txt`. If there are no recoverable input errors, this file is not created. Unrecoverable issues, such as missing required input files, invalid map dimensions, or an unsafe start position, stop the program cleanly with an error message.

The program does not use `exit()`, `abort()`, or similar forced termination. It exits normally through `main`.

4.11 Logging

Logging is controlled by `simulation_config.txt`. When enabled, `simulation_log.txt` records simulation details such as initialization, scans, movement commands, collisions, and final score. Logging is separate from the drone algorithm and does not affect decisions.

5. Testing Approach

5.1 Mock-based testing

The system is designed to run entirely in software. The LiDAR, position sensor, and movement driver are mocks hidden behind interfaces. This allows the drone algorithm to be tested without hardware while preserving the important boundary: the drone receives only sensor-style information

and never the hidden ground-truth map directly.

5.2 Automated tests

The project includes GoogleTest coverage for core behavior. The tests cover map access, mission bounds, out-of-bounds behavior, set/get behavior, and score calculation before running full scenarios.

5.3 Scenario / integration tests

The `edge_cases` scenarios run the compiled `drone_mapper` executable on prepared input directories. They validate complete program behavior, including loading files, running the simulation loop, avoiding collisions, producing `map_output.txt`, and calculating `score.txt`.

Important tested risks include:

Risk	Validation method
Drone collides with a wall or obstacle	<code>MockMovementDriver</code> collision checks and scenario tests.
Drone uses hidden map information	Design review: only mocks, movement validation, and scoring hold the ground-truth map.
Incorrect map states	Unit tests for <code>Map3D</code> and scenario output inspection.
Bad input files	Parser recovery path and <code>input_errors.txt</code> validation.
Missed thin obstacles or blind spots	LiDAR ray-casting and targeted scan behavior in scenarios.
Non-deterministic behavior	Re-running the same scenario should produce the same decisions and output.

5.4 Manual validation

The map files are plain text, so both `map_input.txt` and `map_output.txt` can be inspected manually. The optional logger and `simulation_config.txt` provide command-by-command debugging information without affecting the drone algorithm.

6. Known Limitations

- The implementation assumes the mission resolution is compatible with the provided map resolution, which is allowed for Exercise 1.
- The exploration algorithm prioritizes correctness, safety, and deterministic behavior over optimal path length.
- Exercise 1 does not require returning to the starting position before finishing, so completion is based on reachable frontiers being exhausted.

- Map3D uses a dense flat vector internally rather than a hash-based sparse matrix. This is a deliberate Exercise 1 simplification documented in the design considerations.
- The current scoring implementation compares the discovered map to the ground-truth map, but it does not yet perform a full reachability-based exclusion of completely sealed unreachable areas.